

Drop - A Modern Full-Stack E-Commerce Platform

1. Introduction and Objectives

Drop is a full-stack e-commerce web application designed to provide a modern, responsive, and secure online shopping experience. It covers the complete shopping flow, including product discovery, authentication, cart management, and checkout. The application uses React.js for the frontend, FastAPI for the RESTful backend API, and SQLAlchemy for database management. Its client-server architecture allows the frontend and backend to be developed, tested, deployed, and scaled independently.

The project was undertaken with the following objectives:

1. To design and implement a responsive e-commerce storefront that allows users to browse, search, and view detailed product information.
2. To build a secure user authentication system using JWT (JSON Web Tokens), covering registration, login, and protected route access.
3. To implement core shopping features, including a persistent shopping cart, a wishlist/favorites system, and a structured checkout flow.
4. To design a normalized relational database schema capable of supporting products, users, carts, wishlists, and orders.
5. To apply modern software engineering practices, including RESTful API design, component-based frontend architecture, and clear separation of frontend and backend concerns, throughout development.
6. To deploy the finished application to a live, publicly accessible environment.

2. Problem Statement

Small and mid-sized sellers, as well as students building portfolio-grade projects, frequently need a reference e-commerce platform that is:

- Lightweight enough to run and understand quickly, unlike large commercial platforms such as Shopify and Magento, whose codebases are too large to study or extend for learning purposes.
- Built on a modern, in-demand technology stack, including React, FastAPI, and SQLAlchemy, rather than legacy frameworks, so that the skills learned transfer directly to industry practice.
- Secure by default, with password hashing and token-based authentication rather than ad-hoc session handling.
- Extensible, so that features such as payments, recommendations, and an admin dashboard can be layered on top of a clean foundation instead of being retrofitted into a monolith.

3. System Requirements

3.1 Functional Requirements:

- Users shall be able to register for a new account and log in securely.
- The system shall issue a JWT access token on successful login and use it to authorize subsequent requests.
- Users shall be able to browse products, view product detail pages containing images, price, sizes, category, and description, and search the catalog.
- Users shall be able to add products, with a selected size and quantity, to a shopping cart, update quantities, and remove items.
- Users shall be able to add or remove products from a personal wishlist/favorites section.
- The system shall calculate cart totals automatically as items are added, removed, or updated.
- Users shall be able to proceed to checkout, supplying name, email, shipping address, city, postal code, and payment method.
- The backend shall expose CRUD endpoints for products (GET, POST, PUT, DELETE) so that the catalog can be managed programmatically.
- The system shall support product reviews and ratings.

3.2 Non-Functional Requirements:

- The frontend shall be built with Vite for fast bundling and hot-module reloading during development and shall produce an optimized production build.
- Passwords shall never be stored in plain text; they shall be hashed using Passlib. All protected endpoints shall require a valid JWT.
- The interface shall be fully responsive across mobile, tablet, and desktop viewports.
- The system shall use SQLAlchemy ORM for database abstraction, enabling the use of Neon.tech-hosted PostgreSQL during development and a custom self-hosted PostgreSQL database in production while minimizing changes to the application code.
- The production build shall be deployable to cloud hosting platforms, with Vercel used for the frontend and Render used for the backend, for public access.

3.3 Hardware and Software Requirements:

- **Development environment:**
 - Node.js: Used for the Vite/React frontend.
 - Python 3.13.3: Used for backend development, with pip and a Python virtual environment managed using UV, a Rust-based Python package and project manager.
 - Visual Studio Code: Used as the primary code editor.
 - Git and GitHub: Used for version control and source-code management.

- **Software stack:**

- Frontend Framework: React.js
- Build Tool: Vite
- Styling: Tailwind CSS
- Backend Framework: FastAPI
- ORM (Object-Relational Mapping): SQLAlchemy
- Authentication: JWT + Passlib (password hashing)
- ASGI Server: Uvicorn
- Database: Neon.tech-hosted PostgreSQL for development and a custom self-hosted PostgreSQL database for production
- Deployment: Vercel for the frontend and Render for the backend

4. System Architecture and Design

4.1 High-Level Architecture:

Drop follows a classic three-tier client-server architecture, keeping presentation, application logic, and data storage in separate, independently deployable layers.

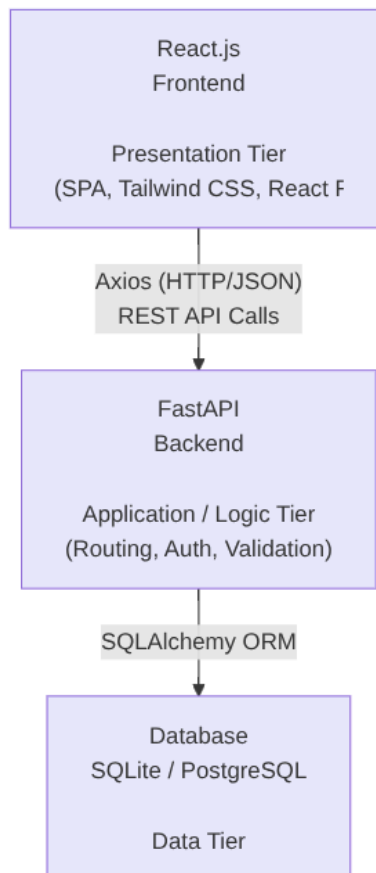


Fig: System Architecture

This separation means the React client only ever talks to the backend through well-defined REST endpoints; it has no direct knowledge of how data is persisted. Conversely, the FastAPI backend has no knowledge of how the UI renders data; it simply returns JSON. This decoupling allows the frontend, deployed on Vercel, and the backend, deployed independently on a Python-friendly host, to evolve, scale, and be redeployed on their own schedules.

4.2 Frontend Architecture:

The frontend is a single-page application (SPA) built with React and Vite. Its responsibilities include:

- Rendering product listings and detail pages from data fetched via Axios.
- Managing client-side UI state, including cart contents, wishlist, and authentication token, so pages feel instantaneous.
- Using React Router DOM for client-side navigation between the storefront, product pages, cart, wishlist, and checkout without full-page reloads.
- Styling with Tailwind CSS, a utility-first CSS framework, to keep the UI consistent and responsive across breakpoints.

4.3 Backend Architecture:

The backend is built with FastAPI, chosen for its performance, built on Starlette and Pydantic, automatic request/response validation, and automatically generated interactive API documentation. Its responsibilities include:

- Exposing a RESTful API under logical resource groupings, such as `/users`, `/products`, `/payments`, and `/orders`.
- Validating incoming request bodies using Pydantic schemas before they reach business logic.
- Hashing and verifying passwords with Passlib and issuing and validating JWT tokens for protected routes.
- Talking to the database exclusively through the SQLAlchemy ORM layer, so the same application code can work with PostgreSQL across development and production environments.
- Serving interactive API documentation automatically at `/docs` (Swagger UI) and `/redoc` (ReDoc).

4.4 Authentication Flow:

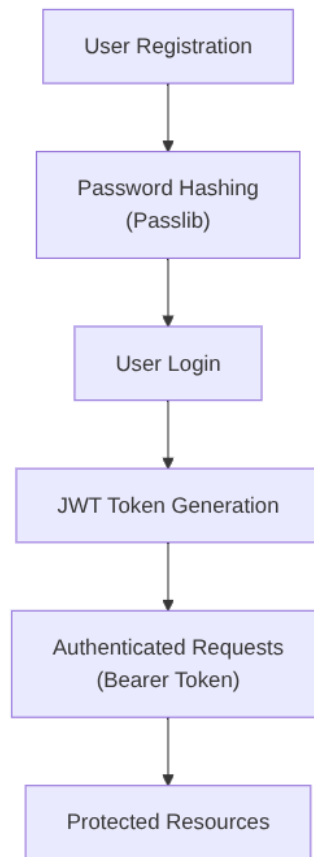


Fig: Authentication Flow

On registration, the plaintext password is never persisted; only its hash is stored. On login, the submitted password is verified against the stored hash. On successful authentication, the server issues a signed JWT that the client attaches to the Authorization header of subsequent requests to protected endpoints, such as cart, wishlist, or order operations.

5. Database Design and ER Diagram

5.1 Design Approach:

The schema is managed through SQLAlchemy models and is designed to be relational and normalized, avoiding duplicated data across tables. The core entities are Users, Products, Cart Items, Wishlist Items, Orders, and Reviews, connected through foreign-key relationships.

5.2 Conceptual Entity–Relationship Diagram:

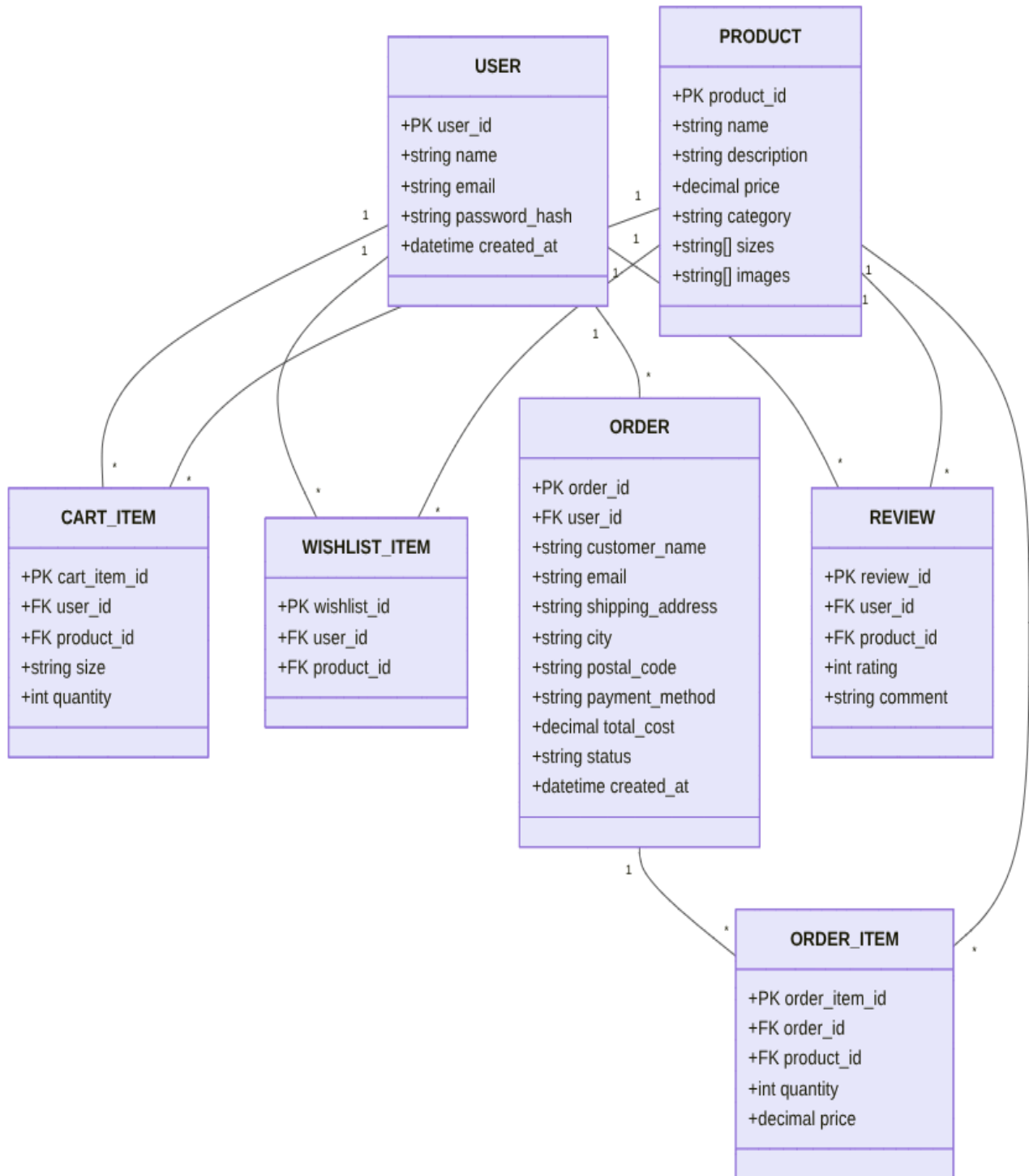


Fig:Class Diagram

Key: 1 - * denotes a one-to-many relationship, such as one User having many Cart Items.

5.3 Entity Descriptions:

Entity	Purpose	Key Attributes
User	Stores registered account data	user_id (PK), name, email (unique), password_hash, created_at
Product	Master catalog record for each item	product_id (PK), name, description, price, category, sizes, images
Cart Item	Line items in a user's active cart	cart_item_id (PK), user_id (FK), product_id (FK), size, quantity
Wishlist Item	Products a user has saved for later	wishlist_id (PK), user_id (FK), product_id (FK)
Order	A placed checkout transaction	order_id (PK), user_id (FK), customer_name, email, shipping_address, city, postal_code, payment_method, total_cost, status
Order Item	Snapshot of each product purchased in an order	order_item_id (PK), order_id (FK), product_id (FK), quantity, price
Review	User ratings/comments on a product	review_id (PK), user_id (FK), product_id (FK), rating, comment

Relationships are one-to-many from User to Cart Item, Wishlist Item, Order, and Review, and one-to-many from Product to Cart Item, Wishlist Item, Order Item, and Review. Order to Order Item is a one-to-many relationship, capturing each product purchased within a single checkout transaction.

6. Description of Major System Modules

6.1 Authentication Module:

Handles user registration, login, password hashing using Passlib, and JWT issuance and validation. It guards protected routes on both the frontend, by redirecting unauthenticated users, and the backend, by rejecting requests without a valid bearer token.

6.2 Product Catalog Module:

Serves the product listing and detail views. It supports search and filtering and returns structured product data, including name, description, images, price, available sizes, and category, which are consumed by the frontend's listing and detail-page components.

6.3 Shopping Cart Module:

Allows an authenticated user to add a product, with a chosen size and quantity, to their cart, update quantities, remove items, and view a running subtotal or total. Cart state is persisted server-side against the user's account through the Cart Item table and mirrored in frontend state for responsive UI updates.

6.4 Wishlist Module:

Lets users bookmark products they are interested in but are not ready to purchase. It is implemented as a simple many-to-many association between users and products.

6.5 Checkout Module:

Collects the information required to place an order, including customer name, email, shipping address, city, postal code, and payment method. On submission, it persists an Order record with associated Order Item rows tied to the authenticated user.

This module is deliberately structured as "payment-ready": the data model and endpoint already capture the information required by a live payment processor, so that a payment gateway can be added without restructuring the frontend checkout flow or order data model.

6.6 Reviews & Ratings Module:

Allows users to leave a star rating and written comment against a product they have purchased or viewed, supporting future features such as average-rating displays and sorted or filtered browsing by rating.

6.7 API/Backend Module

The FastAPI application exposes these resource groups:

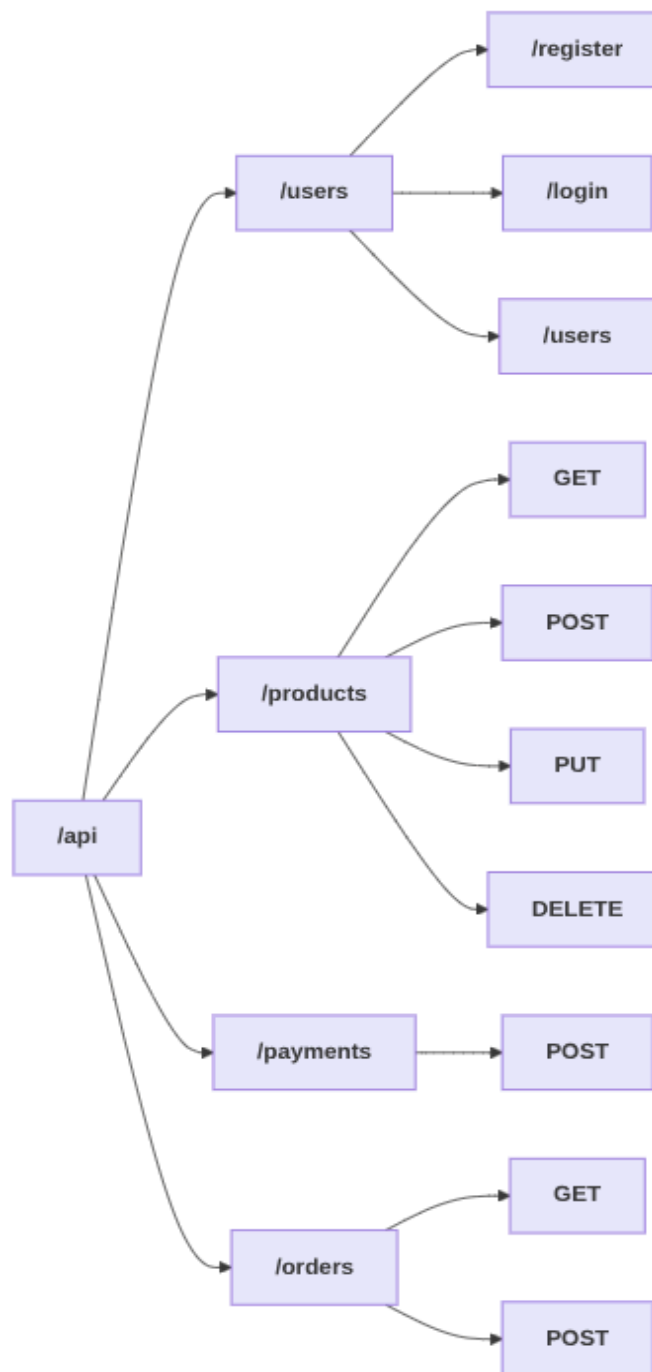


Fig: API Endpoints

Each endpoint validates its payload against a Pydantic schema before interacting with the database. FastAPI automatically generates interactive documentation for the complete API surface at `/docs` and `/redoc`.

7. Payment API Integration

The checkout module was designed with payment integration as a first-class concern rather than an afterthought. The current implementation includes a dedicated `/payments` endpoint and a checkout data model that captures the fields required for payment integration, including customer identity, order total, and selected payment method. This allows a live payment gateway, such as Stripe, PayPal, or eSewa/Khalti for the Nepali market, to be integrated behind the endpoint without requiring major changes to the frontend checkout form or order data model.

At the current stage of development, the payment flow is architecturally payment-ready rather than connected to a live payment gateway. The checkout process captures and stores the intended payment method and order total, but it does not yet process a real transaction through a third-party processor. The system's roadmap lists "Online Payment Gateway Integration" as a planned enhancement, and this report describes it as such rather than overstating its current state.

Planned integration approach:

- On checkout submission, the backend creates an Order in a pending state and calls the chosen payment provider's API to create a payment or checkout session, passing the order total and a reference ID.
- The client is redirected to, or presented with, the provider's secure payment interface, so card or wallet details never pass through Drop's own servers, reducing PCI-DSS scope.
- The provider notifies the backend through a webhook on success or failure. The backend then updates the Order status accordingly, such as paid, failed, or cancelled.
- The frontend polls or is redirected back to an order-confirmation page reflecting the final payment status.

This design keeps payment-processing concerns isolated behind a single endpoint, consistent with the project's overall separation-of-concerns philosophy.

8. Recommendation Approach and Implementation

Personalized product recommendations are listed on the project roadmap as an upcoming feature, including "AI-powered Product Recommendations" and "Personalized Shopping Experience", rather than something shipped in the current build.

This section documents the recommendation approach designed for that future phase so that the data captured today, including views, cart additions, wishlist additions, purchase history, and reviews, is already structured to support it.

8.1 Proposed Approach:

A hybrid strategy is proposed, combining two well-established techniques:

- Recommend products similar to ones a user has viewed or purchased, using shared attributes such as category, price range, and product metadata. This works well even for new users with little interaction history and helps address the cold-start problem.
- Recommend products that other users with similar behavior, such as similar cart, wishlist, or purchase patterns, have engaged with. This captures preferences that attribute similarity alone may miss.

8.2 Why this data model supports it:

Because the schema already records Cart Item, Wishlist Item, Order/Order Item, and Review rows per user, the raw signals needed to train either approach, including implicit feedback such as views, additions, and purchases and explicit feedback such as star ratings, can be captured from the beginning.

No major schema migration would be required to begin building the recommendation feature. The implementation would primarily involve adding an offline scoring job, such as a scheduled Python job using pandas and scikit-learn, to compute item-similarity or user-similarity matrices and a /products/recommended endpoint to surface the results.

8.3 Implementation status:

At the current stage, the storefront does not yet serve model-generated recommendations. Product discovery relies on catalog browsing, search, and filtering, as described in Section 6.2. The above design represents the intended next phase of the recommendation approach and is structured to build directly on the existing schema.

9. Additional Features Implemented

Beyond the core browse-cart-checkout loop, the following features were implemented to round out the shopping experience:

- Product reviews and ratings, allowing users to leave feedback on individual products.
- Fully responsive design, making the storefront usable on mobile, tablet, and desktop without a separate mobile build.
- Fast, modern tooling through Vite-powered development and production builds, providing near-instant hot-module reloading during development.
- Automatic, interactive API documentation generated by FastAPI at /docs and /redoc, useful both for development and for future API integrators.
- Structured, checkout-ready order data capturing customer, shipping, and payment-method details in a single normalized flow.

The project roadmap additionally scopes out features for subsequent phases, including seller/vendor accounts, an admin dashboard, sales analytics, advanced order tracking, email notifications, inventory management, a discount/coupon system, and real-time notifications. These are discussed further in Section 13.

10. Testing and Results

10.1 Testing Approach:

Testing focused on validating the core user journeys end-to-end, exercising both the frontend UI and the underlying REST API:

Test Area	What Was Verified
User registration/login	Duplicate email is rejected; incorrect password is rejected; successful login returns a valid JWT.
Protected routes	Requests to cart/wishlist/order endpoints without a valid token are rejected (401 Unauthorized).
Product catalog	Listing and detailed endpoints return correctly structured product data; search/filter narrows results as expected.
Cart operations	Adding, updating quantity, and removing an item all correctly update the cart total.
Wishlist operations	Adding/removing a product toggles its wishlist membership without affecting the cart.
Checkout	Submitting valid checkout data creates an Order record with the correct linked Order Item rows and total.
API documentation	Swagger UI (/docs) and ReDoc (/redoc) correctly reflect every implemented endpoint and its schema.
Responsiveness	UI was manually verified across mobile, tablet, and desktop breakpoints.

10.2 Tools Used:

- FastAPI's interactive Swagger UI (/docs) was used for manual endpoint-level testing during backend development, sending requests directly against each route and inspecting responses and status codes.

- Browser DevTools, including the Network tab and responsive design mode, were used to verify Axios request and response payloads and responsive breakpoints on the React frontend.

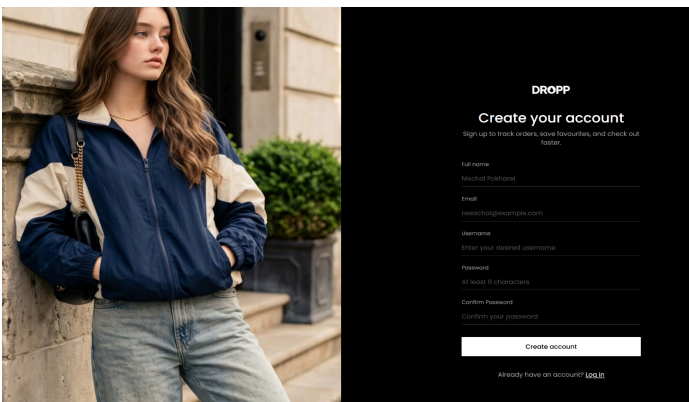
10.3 Results:

All core functional requirements listed in Section 3.1 were verified as working against the deployed build. Registration and login correctly issue and validate JWTs; protected endpoints reject unauthenticated requests; cart and wishlist operations persist and reflect correctly in the UI; and checkout submissions produce correctly linked order records.

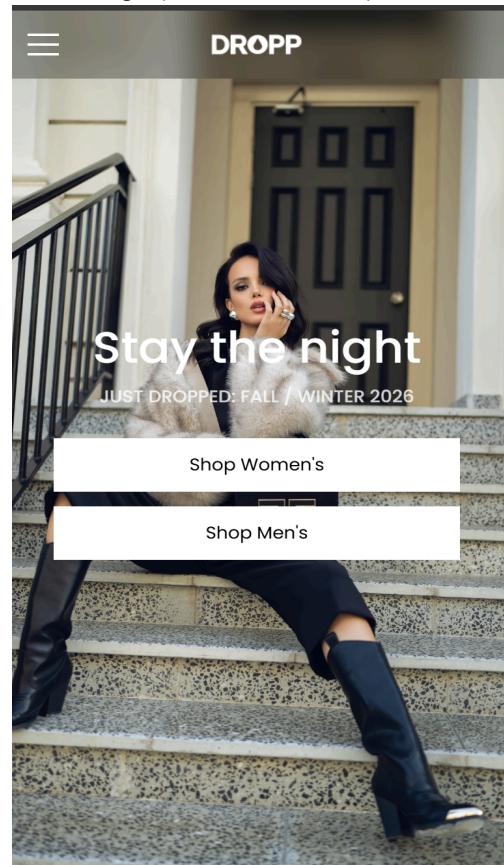
The production frontend build (npm run build / npm run preview) was verified to run successfully and is deployed live at the project's Vercel deployment.

11. Screenshots of the Developed System

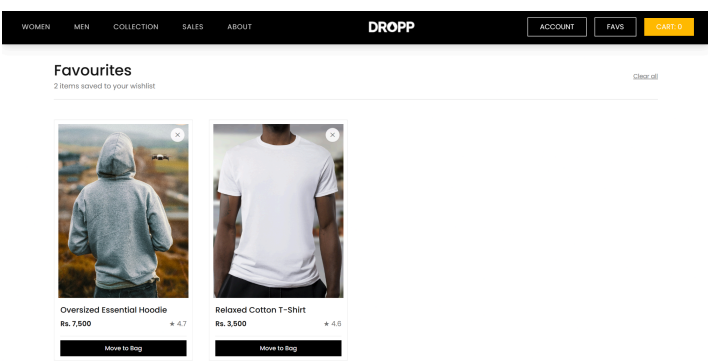
Login/Signup:



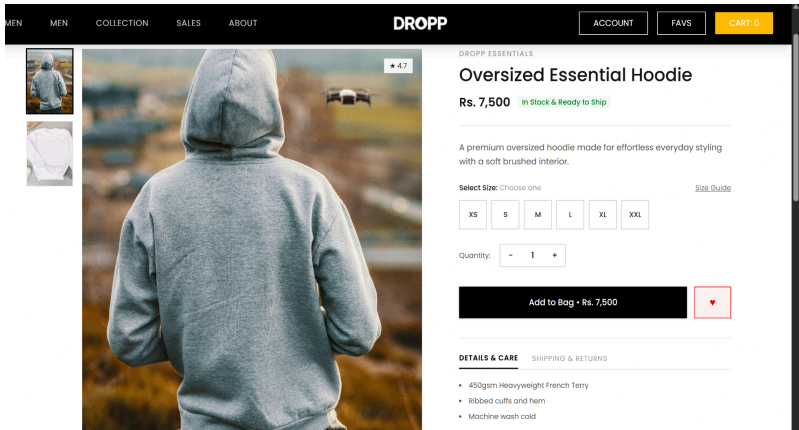
Home Page (from mobile view):



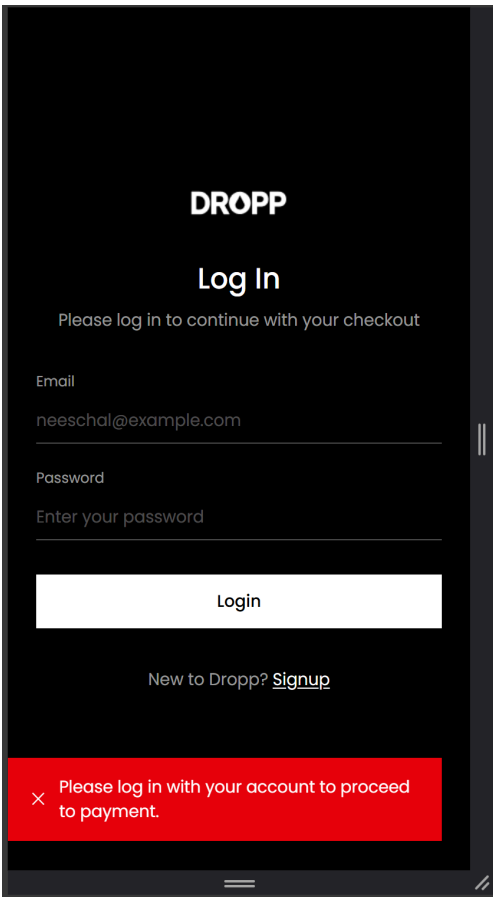
Favourites:



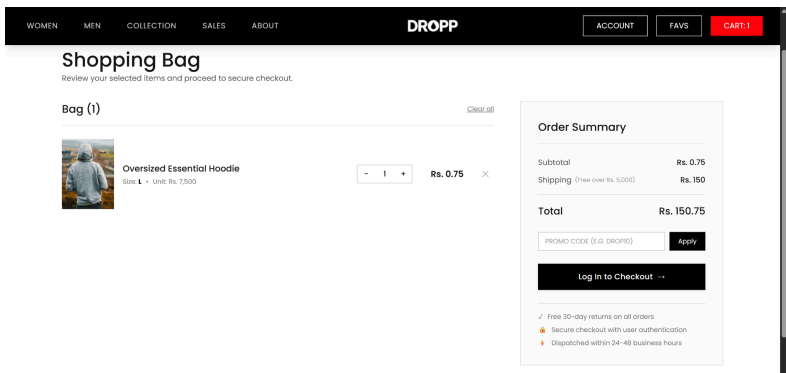
Product Description:



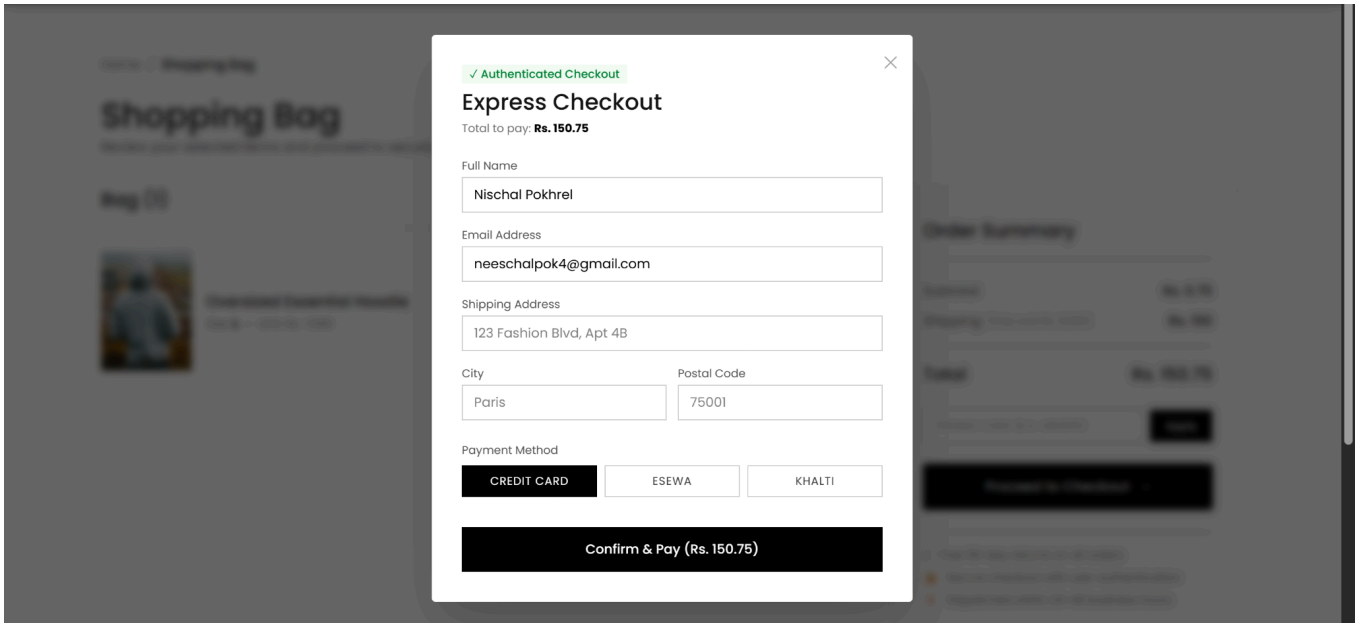
Redirect unauthenticated users to the login page when they try to purchase a product (in a mobile view):



Shopping bag:



Checkout Details:



Payment Details:

Apple Pay

VISA 4242

OR

Email

Continue with Link

neeschalpok04@gmail.com

Payment method

Card

Card information

4242 4242 4242 4242

VISA

12 / 34123

Cardholder name

Neeschal

Country or region

Nepal

Pay

Footer:

Experience the New Standard

Discover our latest drop and build a cohesive, long-lasting wardrobe today.

Shop All Collections

Join the DROPP list

Be first to know about new arrivals, exclusive drops, and offers.

Enter your email

Subscribe

Shop

Women

Men

New Arrivals

Sale

Company

Our Story

Meet the Makers

Careers

Sustainability

Support

Contact Us

FAQs

Shipping & Returns

Size Guide

© 2026 DROPP. All rights reserved.

InstagramTikTokXPinterest

Recommended Products:

WOMENMENCOLLECTIONSALESABOUT

DROPP

NischalFAVSCART: 1

Recommended For You

Handpicked essentials that pair effortlessly

Relaxed Linen Wide Pants

Rs. 7,200

★ 4.7

Essential Boxy T-Shirt

Rs. 3,900

★ 4.8

Oversized Denim Jacket

Rs. 8,900

★ 4.9

Straight Fit Cargo Trousers

Rs. 8,200

★ 4.6

Soft Knit Polo

Rs. 6,800

View all

Team behind Drop:

WOMENMENCOLLECTIONSALESABOUT

DROPP

NischalFAVSCART: 1

BEHIND THE SCENES

Meet the Makers

From patternmakers to textile dyers, each garment passes through skilled hands dedicated to sartorial excellence.

Nischal Pokharel

Frontend Developer

Keshav Prasad Neupane

Backend Developer

12. Challenges Encountered and Solutions

Challenge	Solution
Coordinating two independently running services, the Vite development server on port 5173 and FastAPI on port 8000, during local development, including cross-origin requests from the browser.	Configured CORS on the FastAPI backend to explicitly allow the frontend's origin and centralized the API base URL in a single Axios instance/configuration so environment-specific URLs, local versus production, are switched in one place.
Keeping cart and wishlist state consistent between the frontend's in-memory UI state and the authoritative server-side data.	Treated the backend as the single source of truth. Every cart or wishlist mutation is sent to the API first, and the UI updates from the API response rather than optimistically assuming success.
Validating and securing user input, particularly during registration and checkout, without hand-rolling validation logic.	Used Pydantic schemas on every FastAPI route to declaratively validate request bodies, returning structured 422 errors automatically for malformed input.
Preventing passwords from ever being stored or logged in plaintext.	Adopted Passlib for one-way password hashing at registration time and compared hashes during login rather than handling plaintext passwords.
Designing a checkout flow that could support a real payment gateway later without a schema rewrite.	Modeled the Order and /payments endpoint around the fields required by payment gateways, including total, payment method, and customer/shipping details, as described in Section 7.
Ensuring the storefront looked correct across a wide range of screen sizes without a separate mobile codebase.	Built the UI with Tailwind CSS utility classes and responsive breakpoint modifiers and tested it manually across mobile, tablet, and desktop widths.
Deploying a frontend that depends on a separately running backend.	Deployed the frontend independently to Vercel, with the backend designed to run behind its own URL/host, consistent with the project's client-server separation goal.

13. Conclusion

Drop successfully demonstrates a complete, modern e-commerce workflow, including product discovery, authentication, cart and wishlist management, and a structured, payment-ready checkout, built on a decoupled React/FastAPI/SQLAlchemy stack.

The project met its core objectives by providing a responsive, secure, and extensible storefront backed by a normalized relational schema and a documented REST API. The application was also deployed to a publicly accessible environment.

Where features go beyond the current build, such as live payment processing and AI-driven recommendations, the underlying data model and API surface were deliberately designed to accommodate them without requiring a major rearchitecture. As a result, the project provides a solid foundation for future development rather than functioning as a closed, finished product.

14. Future Enhancements

Per the project's own roadmap, planned next steps include:

- Seller/vendor accounts, allowing multiple merchants on the platform.
- Advanced order management and order tracking.
- Live online payment gateway integration, as described in Section 7.
- Automated email notifications.
- A dedicated user dashboard and admin dashboard.
- Sales and analytics reporting.
- A more advanced review and rating system.
- Improved, faceted product search.
- Discount and coupon codes.
- Inventory management.
- Real-time notifications.
- AI-powered product recommendations and a personalized shopping experience, as described in Section 8.
- Cloud-based image storage.
- Further production hardening and deployment improvements.

15. References

- Drop. *Live Application*. Vercel. <https://dropp-ten.vercel.app/>
- Drop. *Source Code Repository*. GitHub. <https://github.com/keshavprasadneupane/Drop>
- FastAPI. *Modern, high-performance web framework for building APIs with Python*. <https://fastapi.tiangolo.com/>
- JWT.io. *Introduction to JSON Web Tokens*. <https://jwt.io/introduction>
- Passlib. *Password hashing library for Python*. Read the Docs. <https://passlib.readthedocs.io/>
- Pydantic. *Data validation using Python type hints*. <https://docs.pydantic.dev/>
- React. *A JavaScript library for building user interfaces*. <https://react.dev/>
- SQLAlchemy. *The Python SQL Toolkit and Object Relational Mapper*. <https://www.sqlalchemy.org/>
- Tailwind CSS. *A utility-first CSS framework*. <https://tailwindcss.com/>
- Uvicorn. *ASGI web server implementation for Python*. <https://www.uvicorn.org/>
- Vercel. *Frontend cloud platform used for deployment*. <https://vercel.com/>
- Vite. *Next Generation Frontend Tooling*. <https://vitejs.dev/>